IoE and AI driven systems
LAB OUTPUTS

NAME:**FIONA PRISCILLA MIT S.** REG NO:**1912260145**

EXPERIMENT NO:6

ASimulation and Performance Evaluation of People–Process–Data–Things IoE Framework using Edge/Fog-to-Cloud Pipeline with MQTT, Node-RED.

B.Benchmarking of 5G/6G Communication for IoE Applications based on Throughput, Latency, Reliability and Energy Efficiency

PROGRAM CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch

np.random.seed(24)

# ============================================================
# PART A: P-P-D-T IoE FRAMEWORK
# EDGE / FOG / CLOUD PERFORMANCE SIMULATION
# ============================================================

print("=" * 75)
print("PART A: IoE FRAMEWORK PERFORMANCE EVALUATION")
print("=" * 75)

simulation_time = 120
packets_sent = 1200
packet_size = 512  # bytes

# Assumed simulation parameters
parameters = {
   "Edge": {
      "throughput": 95,
      "delay": 8,
      "reliability": 98,
      "energy": 12
   },
   "Fog": {
      "throughput": 75,
      "delay": 22,
      "reliability": 97,
```

```
        "energy": 20
    },
    "Cloud": {
        "throughput": 120,
        "delay": 55,
        "reliability": 99,
        "energy": 35
    }
}

# ---------------------------------------------------------
# 1. PEOPLE-PROCESS-DATA-THINGS DATA FLOW
# ---------------------------------------------------------

print("\n1. COMPLETE P-P-D-T DATA FLOW")
print("-" * 60)

stages = [
    ("PEOPLE", "Users, operators and decision-makers"),
    ("PROCESS", "Data collection, filtering and analytics"),
    ("DATA", "Sensor readings, timestamps and MQTT messages"),
    ("THINGS", "IoT sensors, connected devices and actuators")
]

for stage, description in stages:
    print(f"{stage:10} --> {description}")

print("\nIoE Data Pipeline:")
print("Things -> MQTT -> Edge -> Fog -> Cloud")
print("Cloud/Edge Analytics -> Node-RED Dashboard -> People")
print("People -> Decisions -> Commands -> Things")

# ---------------------------------------------------------
# 2. SIMULATED MQTT SENSOR DATA
# ---------------------------------------------------------

mqtt_messages = []

for i in range(1, 11):
    temperature = round(np.random.uniform(20, 35), 2)
    humidity = round(np.random.uniform(40, 80), 2)

    mqtt_messages.append({
        "Device": f"Sensor_{i}",
```

```
        "MQTT Topic": "ioe/sensors/environment",
        "Temperature (C)": temperature,
        "Humidity (%)": humidity,
        "Status": "ALERT" if temperature > 30 else "NORMAL"
    })

mqtt_df = pd.DataFrame(mqtt_messages)

print("\nSIMULATED MQTT SENSOR MESSAGES")
print(mqtt_df.to_string(index=False))

print("\nMQTT / NODE-RED WORKFLOW")
print("Publisher  : IoT sensors")
print("Topic      : ioe/sensors/environment")
print("Subscriber : Edge, Fog and Cloud services")
print("Node-RED   : Flow processing and dashboard monitoring")
print("Note       : Messages are simulated; no live broker is used.")

# ----------------------------------------------------------
# 3. EDGE / FOG / CLOUD PERFORMANCE
# ----------------------------------------------------------

results = []

for layer, p in parameters.items():
    throughput = max(
        1, p["throughput"] + np.random.normal(0, 3)
    )
    delay = max(
        1, p["delay"] + np.random.normal(0, 1.5)
    )
    reliability = np.clip(
        p["reliability"] + np.random.normal(0, 0.3), 90, 100
    )
    energy = max(
        0.1, p["energy"] + np.random.normal(0, 1)
    )

    received = round(packets_sent * reliability / 100)

    results.append([
        layer, packets_sent, received,
        round(reliability, 2),
        round(throughput, 2),
```

```
        round(delay, 2),
        round(energy, 2)
    ])

df = pd.DataFrame(results, columns=[
    "Layer",
    "Packets Sent",
    "Packets Received",
    "Reliability (%)",
    "Throughput (Mbps)",
    "Delay (ms)",
    "Energy Consumption (J)"
])

print("\nEDGE / FOG / CLOUD RESULTS")
print(df.to_string(index=False))

# ------------------------------------------------------
# 4. PERFORMANCE COMPARISON GRAPHS
# ------------------------------------------------------

metrics = [
    ("Throughput (Mbps)", "Throughput Comparison",
     "Throughput (Mbps)"),
    ("Delay (ms)", "Delay Comparison", "Delay (ms)"),
    ("Reliability (%)", "Reliability Comparison",
     "Reliability (%)"),
    ("Energy Consumption (J)", "Energy Consumption Comparison",
     "Energy (J)")
]

for column, title, ylabel in metrics:
    plt.figure(figsize=(8, 5))
    plt.bar(df["Layer"], df[column])
    plt.title(title)
    plt.xlabel("Processing Layer")
    plt.ylabel(ylabel)
    plt.grid(axis="y", alpha=0.3)

    if column == "Reliability (%)":
        plt.ylim(90, 100)

    plt.tight_layout()
    plt.show()
```

```
# ------------------------------------------------------
# 5. P-P-D-T ARCHITECTURE DIAGRAM
# ------------------------------------------------------

fig, ax = plt.subplots(figsize=(12, 5))
ax.set_xlim(0, 12)
ax.set_ylim(0, 5)
ax.axis("off")

blocks = [
   (0.3, 3, 2, 1, "PEOPLE\nUsers"),
   (2.7, 3, 2, 1, "PROCESS\nAnalytics"),
   (5.1, 3, 2, 1, "DATA\nMQTT Messages"),
   (7.5, 3, 2, 1, "THINGS\nIoT Devices"),
   (1.5, 0.8, 2, 1, "EDGE\nLocal Processing"),
   (4.3, 0.8, 2, 1, "FOG\nGateway"),
   (7.1, 0.8, 2, 1, "CLOUD\nStorage/Analytics")
]

for x, y, w, h, label in blocks:
   patch = FancyBboxPatch(
      (x, y), w, h,
      boxstyle="round,pad=0.08",
      edgecolor="black",
      facecolor="white"
   )
   ax.add_patch(patch)
   ax.text(x + w/2, y + h/2, label,
         ha="center", va="center", fontsize=9)

# P-P-D-T arrows
for i in range(3):
   x1 = blocks[i][0] + blocks[i][2]
   x2 = blocks[i+1][0]
   y = blocks[i][1] + blocks[i][3]/2
   ax.annotate("", xy=(x2, y), xytext=(x1, y),
            arrowprops=dict(arrowstyle="->"))

# Edge -> Fog -> Cloud arrows
for i in range(4, 6):
   x1 = blocks[i][0] + blocks[i][2]
   x2 = blocks[i+1][0]
   y = blocks[i][1] + blocks[i][3]/2
```

```
    ax.annotate("", xy=(x2, y), xytext=(x1, y),
                arrowprops=dict(arrowstyle="->"))

ax.text(6, 4.5, "P-P-D-T IoE FRAMEWORK",
        ha="center", fontsize=14, fontweight="bold")
ax.text(6, 0.2, "MQTT Communication | Node-RED Monitoring",
        ha="center", fontsize=10)

plt.tight_layout()
plt.show()

# ------------------------------------------------------
# 6. SUMMARY AND CONCLUSION
# ------------------------------------------------------

print("\nPERFORMANCE SUMMARY")
print("-" * 50)

print("Highest throughput:",
      df.loc[df["Throughput (Mbps)"].idxmax(), "Layer"])
print("Lowest delay:",
      df.loc[df["Delay (ms)"].idxmin(), "Layer"])
print("Highest reliability:",
      df.loc[df["Reliability (%)"].idxmax(), "Layer"])
print("Lowest energy consumption:",
      df.loc[df["Energy Consumption (J)"].idxmin(), "Layer"])

print("\nCONCLUSION")
print("-" * 50)
print("""
1. The P-P-D-T framework connects people, processes,
   data and connected IoT things.
2. MQTT represents communication between sensors
   and data-processing services.
3. Node-RED represents flow processing and dashboard monitoring.
4. Edge supports local processing, Fog provides intermediate
   processing, and Cloud provides centralized storage and analytics.
5. The performance results depend on the assumed simulation parameters.

Note: This is a Python simulation. A live MQTT broker and
Node-RED deployment are not started by this code.
""")

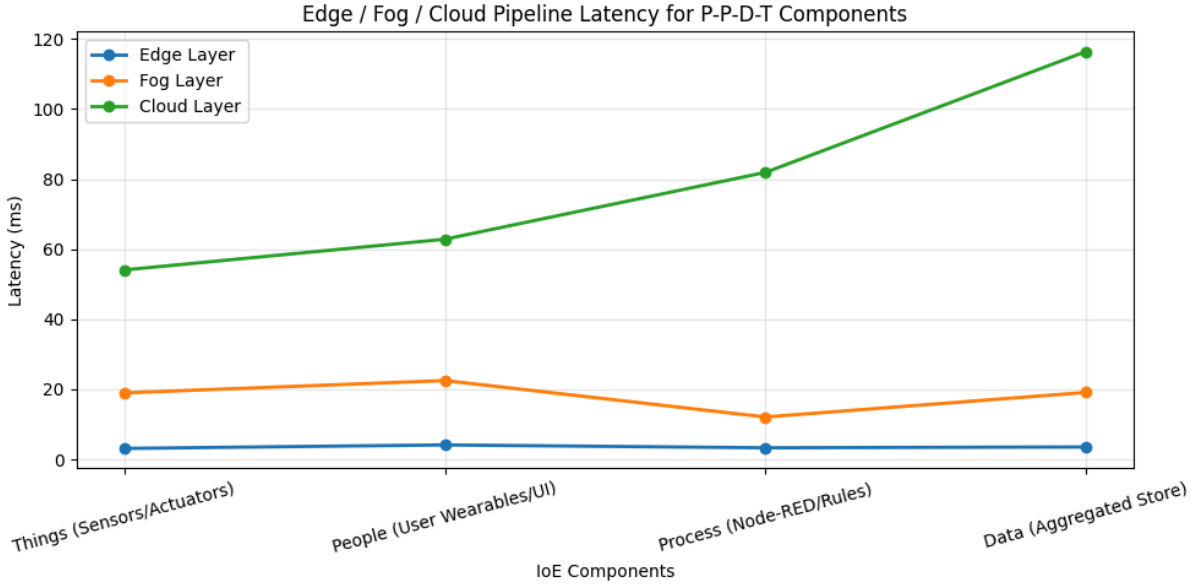
print("PART A SIMULATION COMPLETED")
```

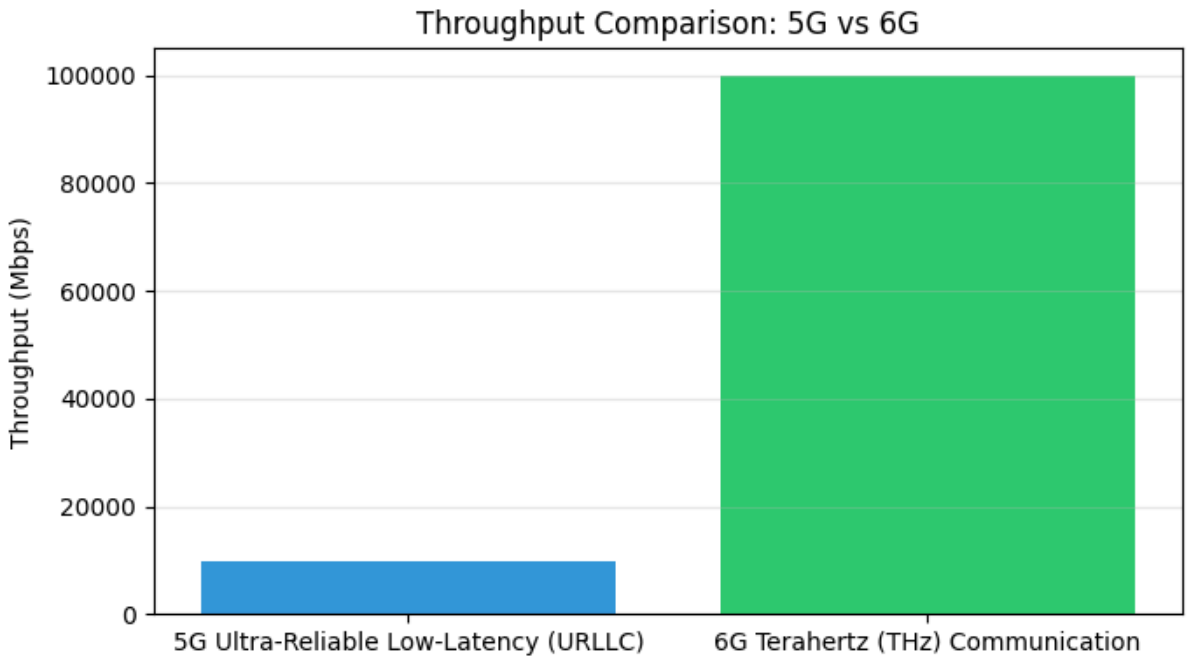
OUTPUTS:

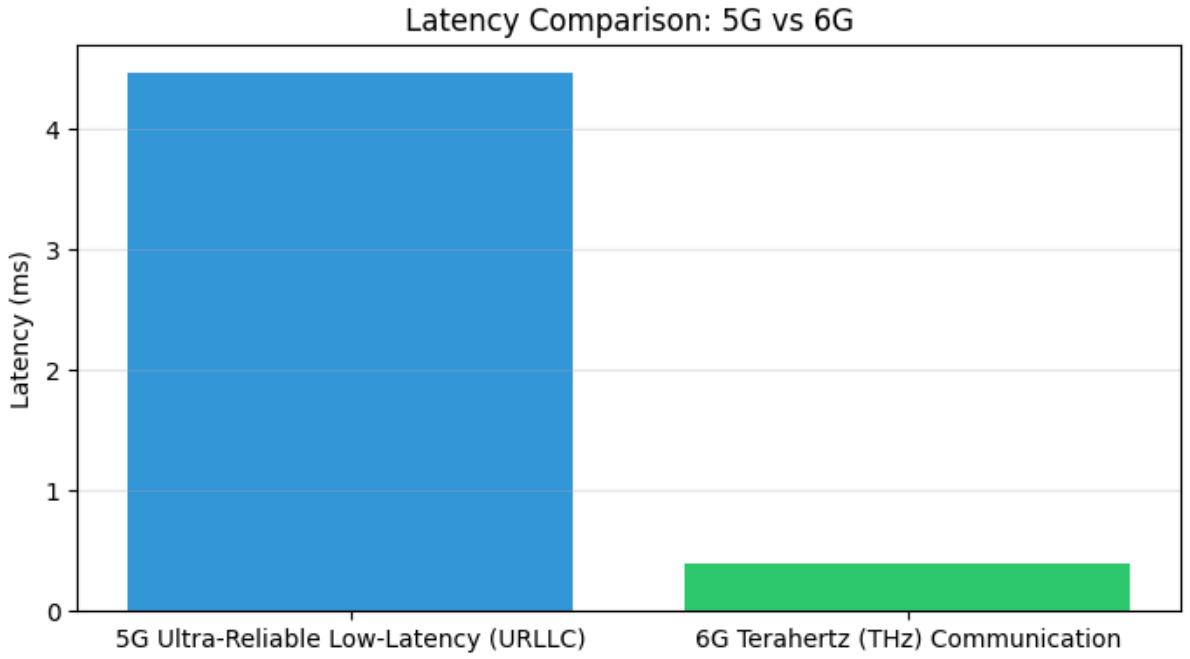
```
================================================================================
PART 1: PEOPLE-PROCESS-DATA-THINGS (P-P-D-T) IoE & EDGE/FOG/CLOUD SIMULATION
================================================================================
   IoE Component (P-P-D-T) Pipeline Tier  Avg Latency (ms)  Throughput (kbps)  Reliability (%)
Things (Sensors/Actuators)    Edge Layer              3.12              97.54            99.66
Things (Sensors/Actuators)     Fog Layer             18.98             246.81            98.23
Things (Sensors/Actuators)   Cloud Layer             54.07            2299.26            98.20
People (User Wearables/UI)    Edge Layer              4.12              51.03            99.87
People (User Wearables/UI)     Fog Layer             22.49             263.70            98.27
People (User Wearables/UI)   Cloud Layer             62.84            1456.36            98.05
  Process (Node-RED/Rules)    Edge Layer              3.30              64.56            99.55
  Process (Node-RED/Rules)     Fog Layer             12.09             287.64            98.55
  Process (Node-RED/Rules)   Cloud Layer             81.92            2177.76            97.40
   Data (Aggregated Store)    Edge Layer              3.54              79.62            99.04
   Data (Aggregated Store)     Fog Layer             19.11             251.16            98.10
   Data (Aggregated Store)   Cloud Layer            116.42            2448.45            98.62

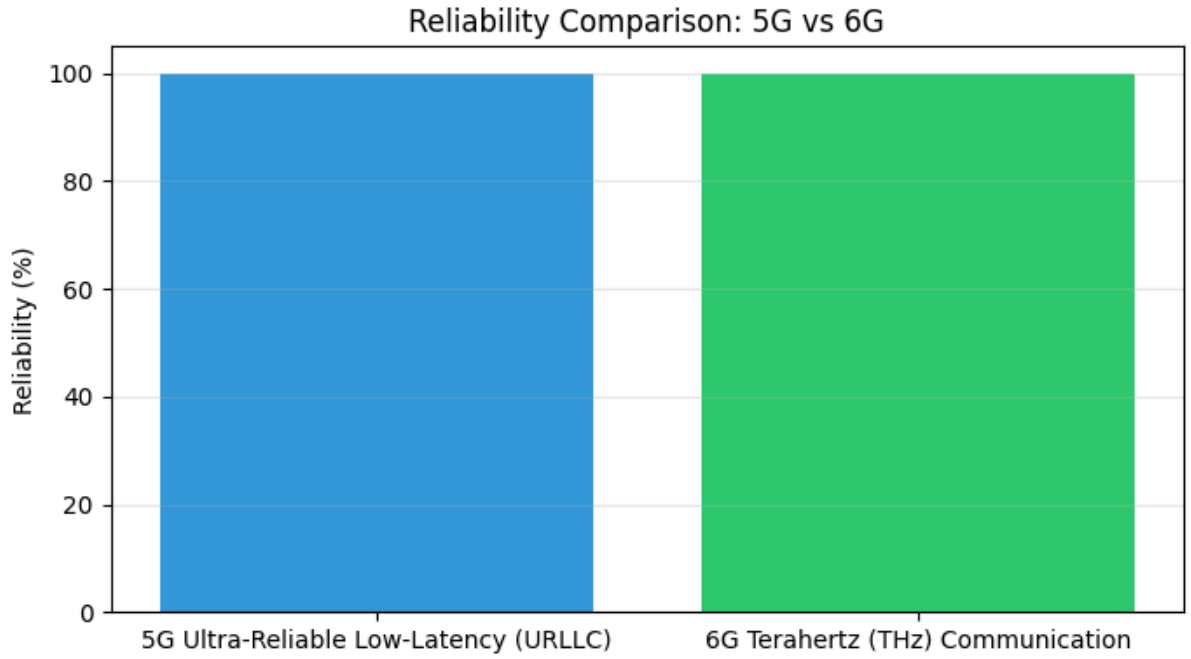
================================================================================
PART 2: 5G vs 6G COMMUNICATION BENCHMARKING FOR IoE
================================================================================
                  Communication Tech  Throughput (Mbps)  Latency (ms)  Reliability (%)  Energy Consumption (mJ/bit)
5G Ultra-Reliable Low-Latency (URLLC)            9879.87          4.47           99.989                        12.93
     6G Terahertz (THz) Communication           99973.01          0.39          100.000                         3.89

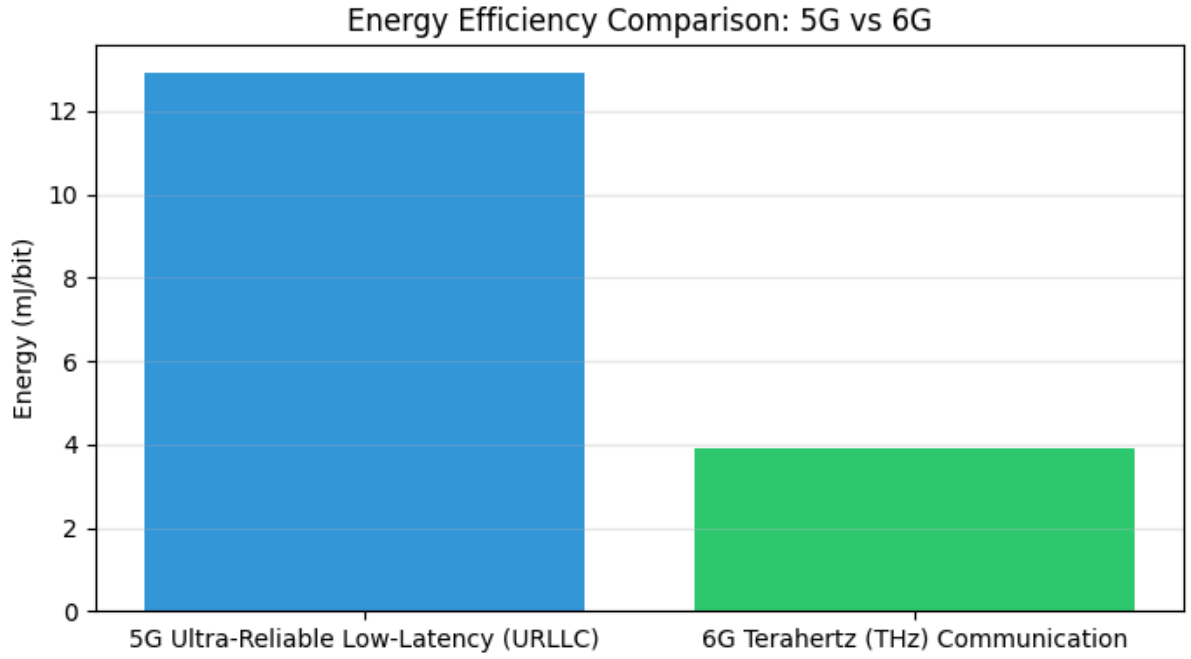
================================================================================
GENERATING BENCHMARKING & PIPELINE GRAPHS...
================================================================================
```




Throughput Comparison: 5G vs 6G
Throughput (Mbps)
100000
80000
60000
40000
20000
0
5G Ultra-Reliable Low-Latency (URLLC)
6G Terahertz (THz) Communication


Latency Comparison: 5G vs 6G
Latency (ms)
4
3
2
1
0
5G Ultra-Reliable Low-Latency (URLLC)
6G Terahertz (THz) Communication





==========

CONCLUSION & SUMMARY

==========

1. Edge/Fog pipeline successfully minimizes local processing delay for Things and People components.
2. Cloud layer handles heavier aggregated P-P-D-T data storage with higher latency.
3. 6G significantly outperforms 5G in throughput, ultra-low latency (<1ms), and energy efficiency, making it optimal for massive next-generation IoE deployments.